0.68.3. Voir la position du A en direct — en simple

Le 0.68 en simple : deplace_croix, puis les deux nombre. Trois lignes dans la boucle.

1. Ce qu'il faut comprendre

C'est le 0.68, en plus simple : la croix tient en une ligne, `deplace_croix` ; il reste les deux `nombre` qui montrent la position.

Trois lignes font tout : bouger, montrer la colonne, montrer la ligne.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <deplace_croix>` → 0.66.3. La fonction deplace_croix() — suivre la croix, case par case
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <nombre>` → 0.8.3. La fonction nombre() — écrire un nombre

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// x et y : voir l'onglet variables.h`	Un commentaire : une phrase pour celui qui lit (« x et y : voir l'onglet variables.h »). La console l'ignore.
`#include <deplace_croix>  // une tuile qui suit la croix, case par case`	Demande à la console « deplace_croix » (ce que c'est : une tuile qui suit la croix, case par case). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui suit la croix, case par case ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur

	refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <nombre>  // écrit un nombre en chiffres`	Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un nombre en chiffres ».)
`#include "variables.h"`	Verse ici le fichier « variables.h » (un autre onglet du programme) : tout se passe comme si son texte était écrit à cet endroit.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`deplace_croix(x, y, ALPHABET[0], 133);  // bouger`	Appel de la fonction deplace_croix de la console, avec les arguments x, y, ALPHABET[0], 133. Ce qu'elle fait : la lettre suit la croix, case par case : à appeler dans la boucle, après image(). vitesse : le temps entre deux pas, en ms écrites en clair (250 = 4 cases par seconde, 100 = 10). Reste dans l'écran, efface seulement si elle a bougé (pas de clignotement). Seule sur sa ligne : deplace_croix(x, y, ALPHABET[0], 250); range la place dans x et y. (Le commentaire le dit : « bouger ».)
`nombre(0, 17, x);  // la colonne, en direct`	Appel de la fonction nombre de la console, avec les arguments 0, 17, x. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la colonne, en direct ».)
`nombre(4, 17, y);  // la ligne, en direct`	Appel de la fonction nombre de la console, avec les arguments 4, 17, y. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la ligne, en direct ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1234 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage         73 o  ← la console, à l'allumage  — la mise en route de la console
  deplace_croix()  217 o  ← main()  — une fonction fournie par le langage
  main()            52 o  ← démarrage  — ta fonction
  VBlank             8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage     28 o  ← image()  — attend l'image suivante
  AttendreVBlank    16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces     12 o  ← EffacerCases, EcrireNombre  — attend de pouvoir écrire une case
  AdresseCase       10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases      12 o  ← effacer()  — efface des cases
  EcrireNombre      33 o  ← nombre()  — écrit un nombre en chiffres
```

```
  CopierTuiles      18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte      14 o  ← démarrage  — vide les deux cartes de l'écran
  LireManette       37 o  ← bouton()  — lit les huit boutons
  Tuiles           704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
  bouton()        × 4    20 o sur place, dans deplace_croix()  + LireManette (37 o)
  nombre()        × 2    20 o sur place, dans main()  + EcrireNombre (33 o)
  effacer()       × 1    15 o sur place, dans deplace_croix()  + AdresseCase (10 o), EffacerCases (12 o)
  poser()         × 1    13 o sur place, dans deplace_croix()  + AdresseCase (10 o)
  image()         × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  images()        × 1     2 o sur place, dans deplace_croix()
Tes fonctions appelées :
  deplace_croix() × 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// x et y : voir l'onglet variables.h

#include <deplace_croix>   // une tuile qui suit la croix, case par case
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <nombre>          // écrit un nombre en chiffres
#include "variables.h"
```

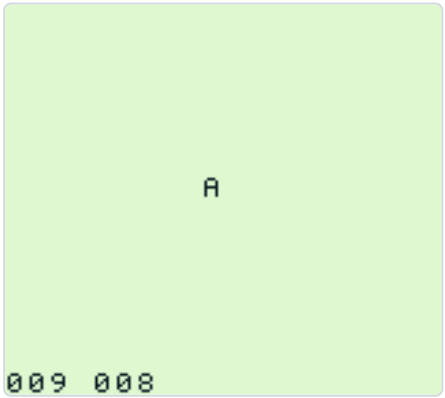
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();
    deplace_croix(x, y, ALPHABET[0], 133);   // bouger
    nombre(0, 17, x);                        // la colonne, en direct
    nombre(4, 17, y);                        // la ligne, en direct
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Comme au 0.68 : le A bouge, sa position s'affiche en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'écran montre la colonne
- ✓ et la ligne

Ensuite : **0.68.4. La fonction lire() — lire une case** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++